



Research Intake 2026

Day 26

Understanding Computer Vision

A Beginner's Guide for Students

Learning Computer Vision Through Images, Videos, and Artificial Intelligence

Gudsky Research Foundation

Table of Contents

1 Introduction to Computer Vision	4
1.1 A Brief History: From "A Summer Problem" to Foundation Models	4
1.2 Why Vision Is a Hard Computational Problem	5
1.3 Module Roadmap: The Twelve-Day Arc	5
2 Digital Image Representation & Color Spaces	6
2.1 Pixels, Bit Depth, and the Image-as-Tensor View	6
2.2 Color Spaces: RGB, Grayscale, HSV, and YCbCr	7
2.3 File Formats: JPEG vs. PNG, and Resolution Considerations	8
2.4 Color Spaces in Practice	9
3 Human Vision vs. Computer Vision	10
3.1 The Biological Visual Pathway	10
3.2 Where the Analogy Breaks Down	10
3.3 Marr's Computational Theory of Vision	11
4 The Computer Vision Task Taxonomy	12
5 The Classical CV Pipeline (Pre-Deep-Learning Era)	13
5.1 Edge Detection: Sobel and Canny	14
5.2 Corner Detection	14
5.3 Handcrafted Feature Descriptors: A Bridge to Day 28	15
5.4 Template Matching	15
5.5 Why Classical Approaches Plateaued, and the Viola-Jones Case Study	15
6 The Deep Learning Shift in Computer Vision	16
6.1 AlexNet and the 2012 ImageNet Moment	16
6.2 Why Convolutions Suit Image Data	16
6.3 Transfer Learning and Pretrained Backbones	17
7 Core CV Libraries & Tooling	17
8 Evaluation Metrics in Computer Vision	19
8.1 Classification Metrics: Accuracy and Top-k Accuracy	19
8.2 Detection Metrics: IoU and mAP	19
8.3 Segmentation Metrics: Pixel Accuracy, Dice, and IoU	20
9 Real-World Applications & Ethical Considerations	20
9.1 Applications Across Industries	20
9.2 Ethical Considerations: Bias, Surveillance, and Consent	21
9.3 A Framework for Responsible Deployment	22
10 Research Frontiers	22
10.1 Foundation Models: CLIP, DINO, and SAM	22
10.2 Vision Transformers: A Structural Alternative to Convolution	22
10.3 Multimodal Models and Self-Supervised Pretraining	23
Chapter Summary: Key Takeaways	23

Assignment / Practical Project Brief.....	25
A Approach A — Classification.....	25
B Approach B — Detection.....	25
C Datasets, Rubric, and Submission.....	26
D Submission Checklist	26

The future of AI is bright, and you can be part of it!

1 Introduction to Computer Vision

Computer vision is the field of artificial intelligence concerned with enabling machines to extract meaningful information from digital images and video — to answer questions such as "what is in this image," "where is it," and "what is happening" from raw pixel data alone. It sits at the intersection of image processing, machine learning, and increasingly, large-scale deep learning, and it underlies an enormous range of modern technology, from the face-unlock on a phone to the perception stack of a self-driving car. Unlike many areas of applied machine learning, where the input is already a clean, structured table of numbers, computer vision begins with an input — a grid of raw pixel intensities — that carries no explicit semantic structure at all; everything from "this is an edge" to "this is a face" has to be recovered from that grid, either by a human-designed algorithm (Section 5) or by a model that has learned to do so from data (Section 6 onward).

1.1 A Brief History: From "A Summer Problem" to Foundation Models

The field's history stretches back further than most people expect, and it is a genuinely humbling one. In the summer of 1966, Seymour Papert at the MIT AI Lab, working under Marvin Minsky, set a group of undergraduate students the task of connecting a camera to a computer and getting the machine to describe what it saw — a project whose internal proposal has since become notorious for badly underestimating the problem's difficulty. That episode is remembered today, only half-jokingly, as the moment the field decided it could "solve vision in a summer." It did not. Progress instead took decades: hand-engineered edge and corner detectors in the 1970s and 1980s, statistical and handcrafted-feature pipelines through the 1990s and 2000s (the subject of Section 5 and the whole of Day 28), and then a genuine inflection point in 2012, when a deep convolutional network called AlexNet dramatically outperformed every prior approach on the ImageNet Large Scale Visual Recognition Challenge. That single result — covered in depth in Section 6 — is conventionally treated as the start of the modern, deep-learning-driven era of computer vision, and it set the field on the trajectory that leads, eventually, to today's large vision and vision-language foundation models, previewed at the end of this day in Section 10 and covered fully in Days 38 through 42 of the broader research programme.

It is worth internalising just how long the "AI winter" gap really was: nearly half a century separates the 1966 Summer Vision Project from AlexNet's 2012 breakthrough. For most of that time, the limiting factor was not a lack of good ideas — convolutional architectures conceptually similar to modern CNNs, such as Kunihiko Fukushima's Neocognitron (1980) and Yann LeCun's LeNet (1989, refined through the 1990s), existed decades before deep learning's mainstream breakthrough. What changed by 2012 was the simultaneous availability of a sufficiently large labelled dataset (ImageNet, with over 14 million labelled images across more than 20,000 categories), sufficiently powerful and affordable parallel compute (consumer GPUs, repurposed from graphics rendering to general-purpose numerical computation), and a handful of training tricks (ReLU activations, dropout regularisation) that made training a deep network on that data and that hardware actually tractable. This is a pattern worth remembering throughout the module: architectural ideas are frequently older than their eventual breakthrough moment, and a breakthrough is often as much a story of data and compute finally catching up as it is a story of a genuinely new idea.

1.2 Why Vision Is a Hard Computational Problem

Vision is a genuinely hard computational problem, and it is worth being explicit about why, since the difficulty shapes almost every design choice covered in this module. A camera records a 2D projection of a 3D world: the same 2D image could, in principle, have been produced by infinitely many different 3D scenes, lighting conditions, and camera positions. Recovering the "real" scene from the image alone is what vision scientists call an inverse problem, and it is ill-posed in the formal mathematical sense — there is no unique solution without additional assumptions. This is directly analogous to trying to recover the two numbers that were added together given only their sum: the equation " $x + y = 12$ " has infinitely many valid solutions, and only additional constraints (x and y must both be positive integers, say) narrow the space down to something tractable.

Human vision resolves this ambiguity effortlessly using a lifetime of prior experience about how the world tends to look — we implicitly assume that surfaces are usually smooth, that objects have consistent size and shape across viewing distance, and that lighting comes predominantly from above. Computer vision systems have to learn (or be given) an equivalent set of priors, whether through hand-designed rules in the classical era (Section 5) or through statistical patterns learned from millions of labelled images in the deep learning era (Section 6 onward). Several concrete sources of ambiguity recur throughout this module and are worth naming explicitly here: scale ambiguity (a small object close to the camera and a large object far away can produce an identical-sized image region), occlusion (objects are very frequently partially hidden behind other objects, and the missing parts must be inferred), illumination variation (the same physical surface reflects very different pixel values under different lighting, as Section 2 discusses), and viewpoint variation (an object looks dramatically different from different angles, yet must still be recognised as the same object) — this last point being exactly the property that the invariant feature descriptors in Day 28 are designed to handle.

Why Vision Is Hard: The Inverse Optics Problem

A single 2D photograph is consistent with an unbounded number of different 3D scenes, surface materials, and lighting arrangements — the projection from 3D world to 2D image throws away depth information irreversibly.

Classical and deep-learning vision systems alike succeed only by learning or encoding additional assumptions (priors) about how real scenes tend to be structured — smooth surfaces, consistent lighting, familiar object shapes — that narrow the space of plausible interpretations down to one.

1.3 Module Roadmap: The Twelve-Day Arc

This module is organised as a twelve-day arc, and it is worth seeing the whole shape of it before diving into the details of any single day, rather than only discovering the arc in Day 37's closing retrospective. Figure 1 lays this roadmap out visually. Days 26 through 28 lay the classical foundations: how images are represented (this day), how they are pre-processed (Day 27), and how features were detected and matched before deep learning existed (Day 28). Days 29 and 30 cover the deep learning shift itself and the architectures that followed from it. Days 31 through 33 apply those learned features to three of the field's

core structured-prediction tasks — object detection, tracking, and pixel-level segmentation. Days 34 and 35 turn the problem around: instead of labelling images, generative models learn to produce new ones, first via GANs and then via diffusion. Day 36 extends everything learned so far into three dimensions and across time (video). And Day 37 closes the module at its highest-stakes application, medical imaging, using the explainability tools needed to trust a model’s decisions.

The Computer Vision Fundamentals Module: A 12-Day Arc

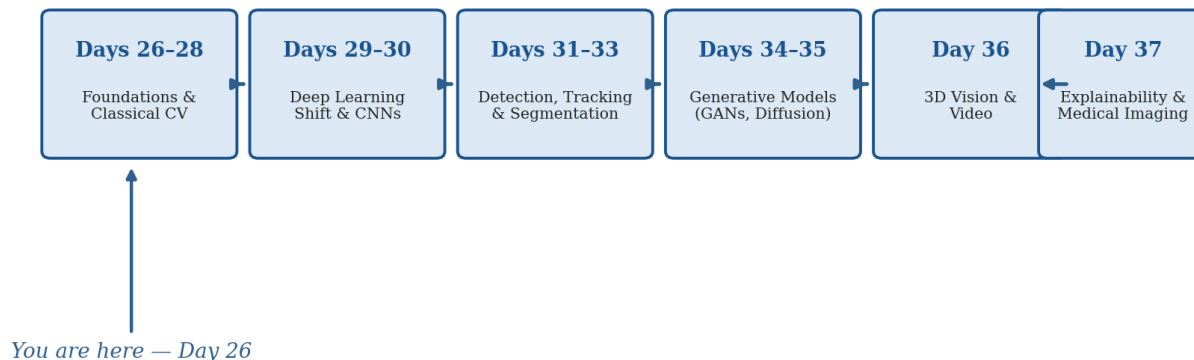


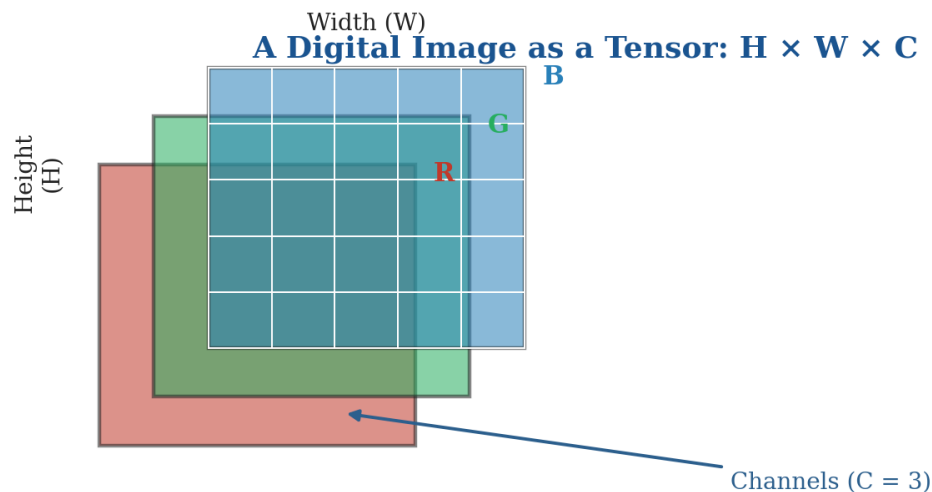
Figure 1. The Computer Vision Fundamentals module’s twelve-day arc, from classical foundations through to explainability and medical imaging.

Each of these stages builds directly on the one before it — Day 28’s feature matching depends on Day 27’s preprocessing choices; Day 29’s CNNs are motivated as a direct response to the limitations of Section 5’s handcrafted features; and the detection, segmentation, and generative-modelling days all depend on the backbone architectures introduced in Days 29 and 30. Keeping this dependency structure in view is the single most useful habit for getting the most out of the material that follows.

2 Digital Image Representation & Color Spaces

2.1 Pixels, Bit Depth, and the Image-as-Tensor View

Every technique covered in this module ultimately operates on the same underlying data structure: a digital image represented as a tensor of shape $H \times W \times C$, where H and W are the image’s height and width in pixels, and C is the number of colour channels, illustrated in Figure 2. A greyscale image has a single channel ($C = 1$), where each pixel stores one intensity value; a standard colour image has three channels ($C = 3$), typically Red, Green, and Blue, stacked as three separate $H \times W$ planes that together describe the full-colour image at every spatial location.



Each cell stores one intensity value, typically an 8-bit integer in the range 0–255 per channel.

Figure 2. A digital colour image represented as a three-channel tensor of shape $H \times W \times C$, with one intensity plane per colour channel.

Each individual pixel value is quantised to a fixed bit depth — most commonly 8 bits per channel, giving 256 possible intensity levels (0–255) per channel, though medical and scientific imaging (relevant again in Day 37) frequently uses higher bit depths such as 12 or 16 bits to preserve finer intensity gradations than an 8-bit image can represent, which matters enormously when a diagnostically relevant feature occupies only a handful of intensity levels near the noise floor of an 8-bit representation.

Worked Example: Storage Cost of Bit Depth

Consider a single colour photograph at 1920×1080 resolution (a common "Full HD" frame size) with 3 channels.

At 8 bits per channel: $1920 \times 1080 \times 3 \times 8 \text{ bits} = 49,766,400 \text{ bits} \approx 6.22 \text{ MB}$ uncompressed.

At 16 bits per channel (common in medical/scientific imaging): $1920 \times 1080 \times 3 \times 16 \text{ bits} \approx 12.44 \text{ MB}$ uncompressed — exactly double, since bit depth scales storage linearly.

This is why lossy compression (JPEG, below) matters so much in practice: a batch of even a few thousand uncompressed 8-bit images already reaches tens of gigabytes, and deep learning training pipelines routinely need to stream through millions of images per epoch.

2.2 Color Spaces: RGB, Grayscale, HSV, and YCbCr

Colour itself can be encoded in more than one coordinate system, and the choice of colour space is a genuine design decision, not just a storage convention — Figure 3 contrasts the two most important spaces geometrically. RGB (Red-Green-Blue) is the default for display and most consumer photography, representing colour as an additive mixture of three primary lights: mixing full-intensity red and green light produces yellow, and mixing all three at full intensity produces white. Grayscale collapses colour entirely, retaining only luminance — useful when colour carries no task-relevant information (many classical edge

and corner detectors, per Section 5, operate on grayscale precisely because gradient structure, not colour, is what they are looking for) and discarding it reduces computation by a factor of three.

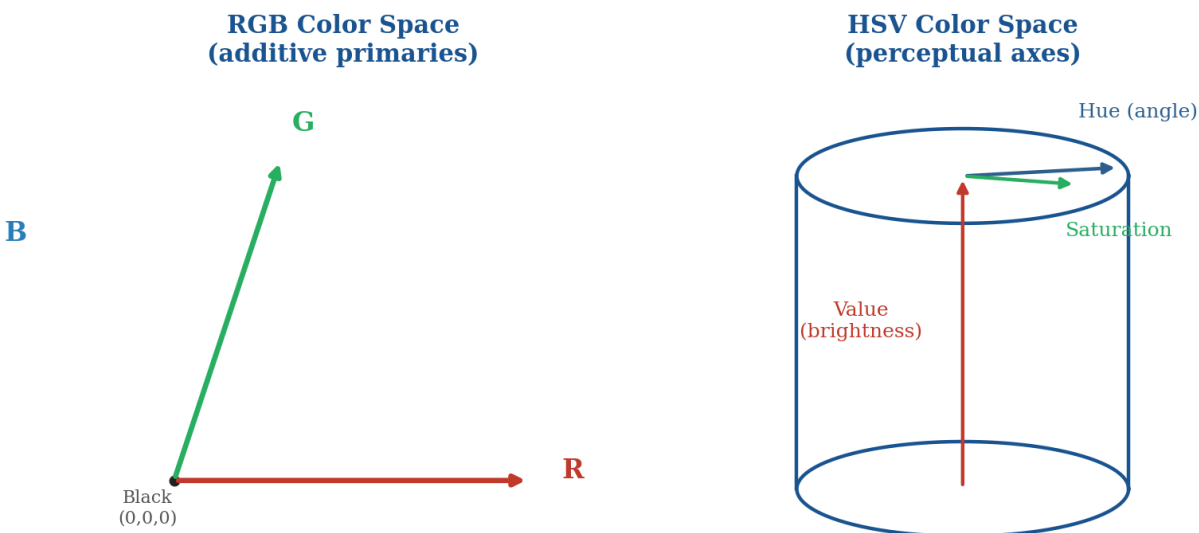


Figure 3. RGB represents colour as a mixture of three additive primaries; HSV re-parameterises colour around the perceptually intuitive axes of hue, saturation, and value.

HSV (Hue, Saturation, Value) re-parameterises colour around perceptually intuitive axes — the hue axis captures "which colour," independent of how saturated or how bright it is — which makes HSV far more convenient than RGB for colour-based segmentation, since a single hue range can isolate "everything red" across a wide range of lighting intensities, a property put to direct practical use in Section 2.4 below. YCbCr separates a luma (brightness) channel, Y, from two chrominance (colour) channels, Cb and Cr, a decomposition that exploits the fact that the human eye is far more sensitive to spatial detail in brightness than in colour, and is the basis for the chroma subsampling used in JPEG compression, discussed next.

2.3 File Formats: JPEG vs. PNG, and Resolution Considerations

File format choice also matters for any pipeline that reads or writes images programmatically. JPEG uses lossy compression: it discards image information — disproportionately in the chrominance channels, following the same brightness-versus-colour sensitivity argument as YCbCr above — to achieve dramatically smaller file sizes, at the cost of visible compression artefacts (blockiness, ringing near sharp edges) that become more pronounced at higher compression ratios. PNG, by contrast, uses lossless compression: the decompressed image is bit-for-bit identical to the original, at the cost of larger file sizes. As a practical rule, PNG (or another lossless format) is preferable for masks, labels, and any image where every pixel value is semantically meaningful — a segmentation mask saved as JPEG can silently corrupt its own class labels at object boundaries, since JPEG's compression artefacts blur exactly the sharp edges a mask depends on — while JPEG is acceptable for natural photographs where minor imperceptible information loss is an acceptable trade for storage efficiency. Resolution and aspect ratio round out the basic representational concerns: a model trained on square, fixed-resolution inputs will need images resized,

cropped, or padded to match — the specific techniques for doing so, and the trade-offs between them, are the subject of Day 27.

2.4 Color Spaces in Practice

Two practical points are worth flagging explicitly here because they trip up nearly every newcomer to the field. First, OpenCV — the most widely used classical computer vision library, introduced properly in Section 7 — loads colour images in BGR channel order by default, rather than the RGB order nearly every other library and every display expects. Feeding a BGR array into a function that expects RGB (or vice versa) does not raise an error; it silently swaps the red and blue channels, producing an image with a visibly wrong but not obviously "broken" colour cast, which makes the bug easy to miss during development.

```
import cv2

# OpenCV loads in BGR order by default -- a very common silent bug source
img_bgr = cv2.imread("photo.jpg")

# Convert to RGB before handing off to any non-OpenCV library
# (PIL, matplotlib, torchvision, ...) which all expect RGB order
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
```

Second, HSV's separation of hue from saturation and value makes it the practical default for any task that needs to isolate objects by colour under variable lighting — for example, segmenting a specific coloured object on a conveyor belt, or detecting traffic signal colours. Thresholding directly in RGB space is fragile, because a colour's RGB triplet shifts substantially as scene brightness changes; thresholding a narrow hue range in HSV is far more robust, since brightness changes are absorbed almost entirely by the value channel and leave hue comparatively unaffected.

```
import cv2
import numpy as np

img_hsv = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2HSV)

# Isolate a red-ish hue range, tolerant of brightness changes
lower_red = np.array([0, 70, 50])
upper_red = np.array([10, 255, 255])
mask = cv2.inRange(img_hsv, lower_red, upper_red)
```

Use Case: Colour-Based Sorting on a Production Line

A produce-sorting robot uses an overhead camera and an HSV colour threshold to separate ripe (red) from unripe (green) tomatoes on a conveyor belt, running entirely on classical OpenCV operations with no deep learning model at all.

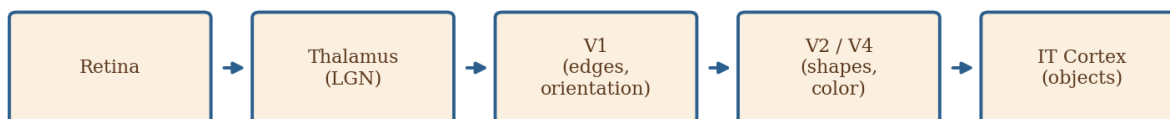
Because the threshold operates on hue rather than raw RGB, the system remains robust across the conveyor's changing overhead lighting throughout the day — a good illustration of why classical techniques (Section 5) remain the right engineering choice for well-defined, constrained problems, even in a field otherwise dominated by deep learning.

3 Human Vision vs. Computer Vision

3.1 The Biological Visual Pathway

The biological visual system provided loose but genuine inspiration for the architectures covered later in this module, and it is useful to understand both the parallel and where it breaks down — Figure 4 lays the two pathways side by side. In the human visual pathway, light striking the retina is converted to neural signals that pass through the thalamus (specifically the lateral geniculate nucleus, LGN) to the primary visual cortex (V1), where neurons respond to simple, localised features — edges and orientations within a small receptive field — before information flows into higher visual areas (V2, V4) that build up increasingly complex, increasingly global representations (textures, colour, shape), eventually reaching inferotemporal (IT) cortex, which supports object recognition. Hubel and Wiesel’s Nobel Prize-winning experiments on cat V1 in the late 1950s and 1960s, which discovered these simple-cell orientation-selective receptive fields, are frequently cited as a direct biological precursor to the local receptive fields and hierarchical feature composition that define convolutional neural networks (Day 29).

Biological Vision (loose inspiration)



Convolutional Neural Network (Day 29)

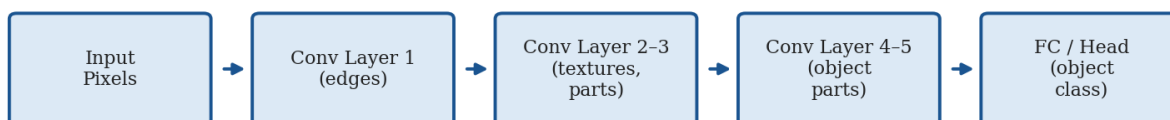


Figure 4. The loose structural parallel between the biological visual pathway and a convolutional neural network’s layer hierarchy — both progress from simple, local features toward complex, global object representations.

3.2 Where the Analogy Breaks Down

The analogy, however, is loose rather than literal, and it is worth being precise about where it breaks down, since overstating it is a common and misleading simplification. CNNs are trained end-to-end via backpropagation and gradient descent on labelled data — a learning mechanism with no known biological analogue in the visual cortex, which instead develops through a mix of genetically specified structure and unsupervised, local learning rules over years of visual experience. CNNs also process an entire image in a single, fixed feed-forward pass through a static architecture, whereas biological vision is an active, temporally extended process involving eye movements (saccades), attention, and substantial top-down feedback from higher cortical areas back to lower ones — feedback pathways that standard feed-forward CNNs simply do not have. Biological vision is also embedded in a body: depth cues from binocular disparity (two eyes), motion parallax from head movement, and the ability to physically reach out and touch an object

to resolve visual ambiguity are all sources of information a static, single-camera computer vision system does not have access to.

Adversarial examples provide some of the clearest empirical evidence for how large this gap actually is. A carefully constructed, often visually imperceptible perturbation added to an image can cause a state-of-the-art CNN to confidently misclassify an object that a human would identify correctly without hesitation — a failure mode with no clear human perceptual analogue. This tells us something important: despite the loose architectural inspiration from biological vision, CNNs are learning something that overlaps with, but is not identical to, whatever representation the human visual system uses, and their failure modes can be strange, brittle, and non-human in character even when their everyday accuracy looks impressively human-like.

Adversarial Examples: A Non-Human Failure Mode

Szegedy et al. (2013) first demonstrated that adding a small, carefully computed, near-imperceptible perturbation to an image could cause a well-trained CNN to confidently misclassify it — a stop sign misread as a speed-limit sign, or a panda misread as a gibbon — with the perturbation invisible or nearly invisible to a human observer.

This is treated throughout this module as a standing caution against assuming CNN "understanding" is human-like just because everyday accuracy is high — a caution that becomes directly practical again in Day 37's discussion of trust and explainability in medical imaging.

3.3 Marr's Computational Theory of Vision

David Marr's computational theory of vision, developed in the late 1970s and published posthumously in 1982, remains a useful conceptual framework for thinking about any vision system, biological or artificial. Marr argued that a visual (or any information-processing) system should be understood at three distinct, complementary levels: the computational level (what problem is being solved, and why — for example, "recover the 3D shape of a surface from its shading"), the algorithmic level (what representations and procedures solve it — for example, "represent the surface as a height map and update it via gradient descent on a shading-consistency loss"), and the implementational level (how it is physically realised — in neurons, or in silicon, or in a specific PyTorch tensor operation).

Marr's Level	Question Asked	Computer Vision Example
Computational	What problem is being solved, and why?	"Assign every pixel a class label" (semantic segmentation, Day 33)
Algorithmic	What representation and procedure solves it?	An encoder-decoder network with skip connections (U-Net, Day 33)
Implementational	How is it physically realised?	A specific CUDA kernel executing convolution on a GPU

This framing is a useful discipline throughout the rest of this module: when a new architecture or technique is introduced, it is worth asking which of Marr's three levels it is actually addressing, since a change at the

implementational level (a faster GPU kernel) and a change at the computational level (redefining the task itself, as Section 4's task taxonomy does) are very different kinds of progress, and conflating them is a common source of confusion when comparing two papers that appear, superficially, to be solving "the same problem."

4 The Computer Vision Task Taxonomy

"Computer vision" is not a single task but a family of related tasks that differ in exactly what kind of output is expected from an input image, and precisely locating a technique within this taxonomy is the first step toward understanding what problem it actually solves. Figure 5 and the table below lay out the standard taxonomy, ordered roughly from coarsest to finest-grained output; nearly every day for the remainder of this module (Days 29 through 35 in particular) can be located as a deep dive into one or more rows of this table.

The Computer Vision Task Taxonomy at a Glance

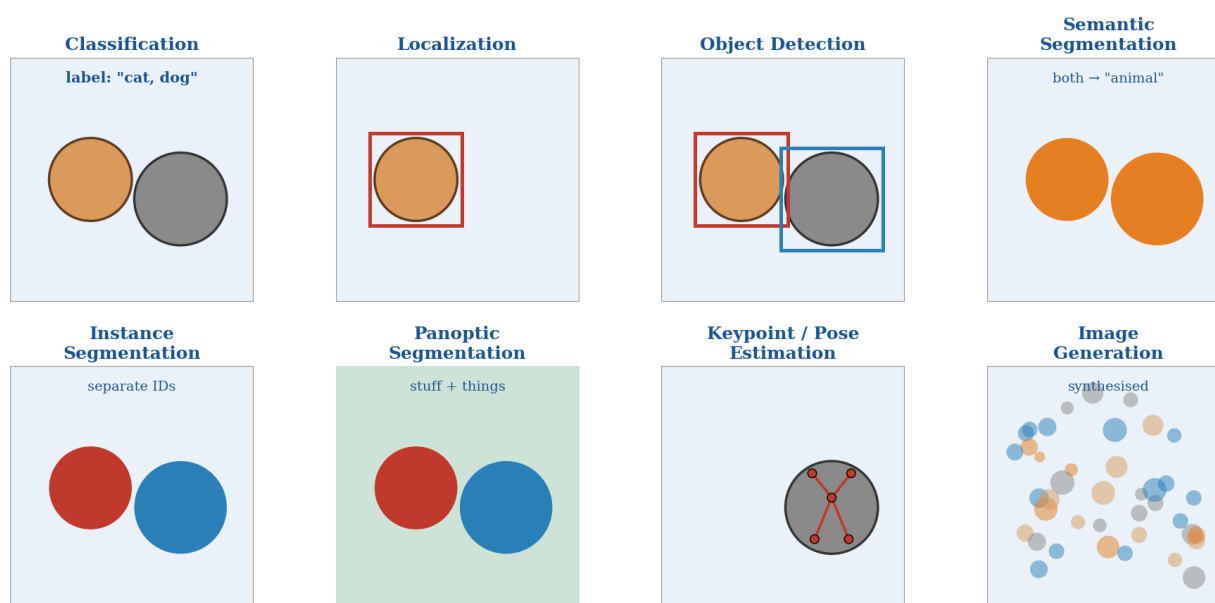


Figure 5. The computer vision task taxonomy illustrated on the same simple scene — the same two objects, but a different kind of output expected at each level of granularity.

Task	Output	Example Question Answered
Image Classification	A single label for the whole image	"Is there a cat in this image?"
Object Localization	One label + one bounding box	"Where is the cat?"
Object Detection	Labels + boxes for every object	"What objects are here, and where is each one?" (Day 31)
Semantic Segmentation	A class label for every pixel	"Which pixels belong to 'cat' vs. 'background'?" (Day 33)

Task	Output	Example Question Answered
Instance Segmentation	A pixel mask per individual object	"Which pixels belong to this specific cat vs. that other cat?" (Day 33)
Panoptic Segmentation	Unified semantic + instance masks	"Label every pixel, and separate individual objects." (Day 33)
Keypoint / Pose Estimation	Coordinates of defined landmark points	"Where are this person's joints?"
Image Generation	A newly synthesised image	"Produce a plausible new image." (Days 34–35)



Worked Example: Choosing the Right Task for a Deployment Scenario

Scenario: A warehouse wants to count how many boxes of each product type are on a shelf, from an overhead camera.

Wrong fit: Image classification — it can only say "boxes are present," not how many or where.

Also insufficient: Semantic segmentation alone would merge all boxes of the same product into one blob, losing the count.

Correct fit: Object detection (Day 31), since it is the row of the taxonomy that natively produces one labelled box per individual object — exactly what a per-item count requires.

This kind of "which row of the table actually matches the business requirement" reasoning is worth practising explicitly — a surprising fraction of real-world computer vision project failures trace back to selecting the wrong task formulation up front, not to a weak model.

This taxonomy is a useful map, but it is worth flagging up front that the boundaries between its rows have become considerably blurrier in the last few years than the clean table above suggests. Foundation models such as the Segment Anything Model (SAM), previewed briefly in Section 10 and covered in depth in Day 33, can perform what looks like detection, semantic segmentation, and instance segmentation within a single promptable architecture, rather than requiring a separate model trained for each row of the table. This is a genuine and important shift in how the field organises itself, but it does not remove the value of the taxonomy — a unified model still has to internally solve each of these sub-problems, and understanding them individually, as this module does across Days 29 through 33, remains the clearest path to understanding what a unified model is actually doing.



Research Spotlight

5 The Classical CV Pipeline (Pre-Deep-Learning Era)

Before deep learning, essentially every computer vision system followed some variant of the same broad pipeline, shown in Figure 6: apply a hand-designed algorithm to detect low-level image structure, extract a compact numerical description of that structure, and feed the result into a comparatively simple classical

machine learning classifier. This section surveys the low-level structure-detection stage of that pipeline; the description and matching stages that follow directly from it are the entire subject of Day 28.

The Classical (Pre-Deep-Learning) Computer Vision Pipeline

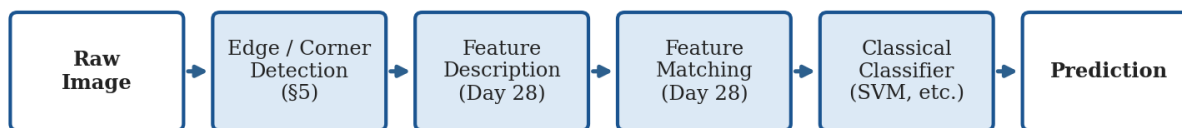


Figure 6. The classical, pre-deep-learning computer vision pipeline: detect structure, describe it numerically, match or classify it.

5.1 Edge Detection: Sobel and Canny

Edge detection identifies pixels where image intensity changes sharply — the boundaries between objects, or between an object and its background. The Sobel operator approximates the image gradient using a pair of small 3×3 convolution kernels (one sensitive to horizontal intensity change, one to vertical), and the gradient magnitude at each pixel gives a simple edge strength map. The Canny edge detector, still widely used today as the classical gold standard, refines this into a complete, multi-stage algorithm: Gaussian smoothing to suppress noise before gradient computation, gradient magnitude and direction computation (frequently using Sobel kernels internally), non-maximum suppression to thin wide gradient responses down to single-pixel-wide edges, and a two-threshold hysteresis step that links weak edge responses into strong ones only where they connect to an already-confirmed strong edge, producing clean, connected edge maps rather than a noisy scattering of gradient responses.

Worked Example: Sobel Gradient on a Simple 3×3 Patch

Consider a small greyscale patch where intensity jumps sharply from left (dark, value 10) to right (bright, value 200) — a clean vertical edge.

Horizontal Sobel kernel $G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$ applied to this patch produces a large positive response, since it sums the weighted difference between the right and left columns.

Vertical Sobel kernel G_y applied to the same patch produces a near-zero response, since intensity is constant along each row.

Gradient magnitude $= \sqrt{G_x^2 + G_y^2}$ is therefore large — correctly flagging this location as an edge, and correctly identifying it as a vertical edge (large G_x , near-zero G_y).

5.2 Corner Detection

Corner detection goes a step further than edges: a corner — a point where intensity changes sharply in more than one direction simultaneously — is a far more distinctive, more reliably re-locatable landmark than a point along a straight edge, which looks identical to every other point along the same edge (a phenomenon

known as the "aperture problem": a small window sliding along a straight edge cannot tell how far it has moved). This distinctiveness is precisely what makes corners the natural building block for the keypoint detection and matching pipeline that Day 28 develops in full, beginning with Harris corner detection and the "Good Features to Track" (Shi-Tomasi) criterion.

5.3 Handcrafted Feature Descriptors: A Bridge to Day 28

Handcrafted feature descriptors extend this idea from "detect an interesting point" to "describe that point's local neighbourhood with a numerical signature robust enough to be recognised again under a different viewpoint, scale, or lighting condition." SIFT (Scale-Invariant Feature Transform) and HOG (Histogram of Oriented Gradients) are the two most historically significant such descriptors, and they form the direct bridge into Day 28 §3 (Classical Keypoint Descriptors), where their construction, comparison, and eventual displacement by learned features are covered in full technical detail.

5.4 Template Matching

Template matching represents a conceptually simpler classical approach: rather than detecting and describing distinctive local structure, a small reference image (the "template") is slid across every position of a larger search image, and a similarity score (typically normalised cross-correlation) is computed at each position, with strong matches indicating a likely location of the template pattern. Template matching is fast and simple but brittle — it has essentially no built-in robustness to rotation, scale change, or partial occlusion of the target, since it is comparing raw pixel intensities directly rather than an invariant description of them.

5.5 Why Classical Approaches Plateaued, and the Viola-Jones Case Study

Collectively, these classical approaches plateaued for a clear reason: hand-designed features and hand-designed invariances can only go as far as a human designer's intuition about which patterns matter, and that intuition breaks down for the enormous natural variability of real-world images — the exact problem that learned features (Day 29 onward) were eventually able to solve by extracting the relevant patterns directly from data rather than from a human's prior assumptions.

The Viola-Jones face detector (2001) is worth understanding as historical context specifically because it is directly relevant to this module's first practical project. It combined simple Haar-like rectangular features (fast-to-compute differences between the summed pixel intensities of adjacent rectangular regions), an AdaBoost-based feature selection process that automatically picked the small subset of Haar features most discriminative for "face vs. not-face," and a cascade of increasingly strict classifiers, arranged so that the vast majority of non-face image regions could be rejected after evaluating only the first, cheapest few features in the cascade. This cascade design achieved, for its time, remarkably fast and reasonably accurate real-time face detection, and it remained the dominant practical face-detection algorithm for over a decade, appearing in real consumer camera autofocus systems long before deep-learning-based face detectors became standard. Its cascade design, one of speed through early rejection of obviously-non-face regions, is conceptually echoed in some of the detection architectures covered later in this module (Day 31), and it is one of the two face-localisation options explicitly available for this day's own practical project, PP12.

Viola-Jones in the Wild

The Viola-Jones detector could process a 384×288 image in roughly 0.067 seconds on a 700MHz Pentium III (2001-era hardware) — a genuinely remarkable speed for its time, achieved almost entirely through the cascade’s early-rejection design rather than raw hardware power.

Its combination of speed and reasonable accuracy meant it shipped in real consumer products — point-and-shoot camera autofocus-on-face features throughout the mid-to-late 2000s were very commonly Viola-Jones-based — making it one of classical computer vision’s clearest commercial success stories.

6 The Deep Learning Shift in Computer Vision

6.1 AlexNet and the 2012 ImageNet Moment

In 2012, a deep convolutional neural network called AlexNet, designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, entered the ImageNet Large Scale Visual Recognition Challenge and won by a startling margin — reducing the top-5 classification error rate from roughly 26% (the best prior, largely classical, approach) to roughly 15%. This was not an incremental improvement; it was a step-change large enough to immediately redirect the field’s research effort away from hand-engineered features of the kind covered in Section 5, and toward learned, end-to-end deep architectures. This single result is conventionally treated as the beginning of the modern deep-learning era of computer vision referenced in Section 1, and its architectural and mechanical details are the entire subject of Day 29.

Approach (ImageNet 2012)	Top-5 Error Rate	Feature Type
Best classical (hand-engineered features + SVM)	$\approx 26.2\%$	Handcrafted (SIFT/Fisher Vectors)
AlexNet (deep CNN, learned features)	$\approx 15.3\%$	Learned end-to-end

6.2 Why Convolutions Suit Image Data

Understanding why convolutional networks in particular suit image data — rather than, say, a generic fully-connected network applied to a flattened pixel vector — comes down to three structural properties that align naturally with how visual information is organised. Local receptive fields mean each unit in a convolutional layer only looks at a small spatial neighbourhood of the input, matching the observation that a pixel’s relevant context is almost always nearby, not on the opposite side of the image. Weight sharing means the same small set of filter weights is applied at every spatial position, which both drastically reduces the number of learnable parameters relative to a fully-connected layer and encodes a sensible assumption directly into the architecture: a useful pattern (an edge, a texture) is just as useful to detect in the top-left corner of an image as in the bottom-right. And this weight sharing directly produces translation invariance (or more precisely, translation equivariance within the convolutional layers themselves) — a cat detector

that has learned to recognise a cat in one part of an image will recognise the same pattern of pixels appearing anywhere else in the image, without needing to separately learn that pattern at every possible location.

Worked Example: Parameter Count: Fully-Connected vs. Convolutional

Consider a single layer processing a modest $224 \times 224 \times 3$ input image, producing 64 output feature maps.

Fully-connected layer: every input pixel connects to every output unit — $224 \times 224 \times 3 \times 64 \approx 9.6$ million weights, for a single layer, before even considering a second layer.

Convolutional layer (5×5 kernel): weight sharing means only $5 \times 5 \times 3 \times 64 = 4,800$ weights are needed — roughly 2,000× fewer parameters for a comparable operation, because the same small kernel is reused at every spatial location rather than learned independently for each.

This parameter efficiency is not just a storage convenience — it is a large part of why CNNs can be trained effectively at all on realistic dataset sizes, since a fully-connected network of comparable depth would need vastly more labelled data to avoid overfitting its far larger parameter count.

6.3 Transfer Learning and Pretrained Backbones

The practical consequence of AlexNet's success that matters most for how the field actually works today is transfer learning: a network pre-trained on a very large, general dataset (ImageNet's 1.2 million labelled images across 1,000 categories being the classic example) learns feature representations general enough to be reused, with comparatively little additional labelled data, as the starting point for an entirely different downstream task. Rather than training a new network from randomly initialised weights for every new problem — which typically requires far more labelled data and compute than most projects have available — practitioners overwhelmingly start from a pretrained backbone and fine-tune it, a pattern this module returns to repeatedly, including directly in this day's own practical project (PP12). The specific architectural mechanics that make all of this possible — what a convolution actually computes, how pooling and activation functions fit together, and how such a network is trained — are covered in full in Day 29, with Day 30 following immediately after to survey how these foundational ideas evolved into today's more sophisticated backbone architectures.

Use Case: Transfer Learning in Practice

A team with only 500 labelled X-ray images of a rare condition cannot realistically train a deep CNN from scratch — 500 images is orders of magnitude too few.

By starting from an ImageNet-pretrained ResNet backbone and fine-tuning only the final layers on their 500 images, teams routinely achieve strong diagnostic accuracy — the general-purpose edge, texture, and shape features learned from 1.2 million natural images transfer surprisingly well even to a visually very different domain like radiography, a theme returned to directly in Day 37.

7 Core CV Libraries & Tooling

A small number of libraries account for the overwhelming majority of applied computer vision work, and knowing which tool is the right fit for a given task is a practical skill in its own right. OpenCV (Open Source Computer Vision Library) is the field's longest-standing and most comprehensive classical computer vision library, providing highly optimised implementations of everything covered in Section 5 (edge and corner detection, filtering) and Day 28 (feature detection and matching), along with a full video I/O and camera-capture interface that makes it the natural choice for any pipeline that needs to read from or write to a live video stream or webcam. PIL/Pillow is a simpler, more lightweight Python imaging library, well suited to basic image manipulation tasks — loading, resizing, cropping, format conversion, simple drawing — where OpenCV's much larger surface area and BGR-ordering quirk (Section 2.4) are unnecessary overhead.

For deep-learning-specific work, torchvision (PyTorch's companion vision library) and the timm library (PyTorch Image Models) provide the field's standard interface to pretrained backbones and their associated pre-processing transforms — precisely the pretrained models that Section 6.3's transfer-learning discussion depends on in practice.

```
import torchvision.models as models
import torchvision.transforms as T

# One line to get an ImageNet-pretrained backbone (Section 6.3)
backbone = models.resnet50(weights="IMAGENET1K_V2")

# The matching preprocessing pipeline the backbone expects
preprocess = T.Compose([
    T.Resize(256),
    T.CenterCrop(224),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406],
                 std=[0.229, 0.224, 0.225]),
])
```

timmm in particular has become the de facto community repository of pretrained architectures spanning the classical CNN lineage (Day 30) through modern vision transformers (Day 38), maintained with a consistency of interface that makes swapping one backbone for another, for benchmarking purposes, a matter of changing a single string argument rather than rewriting model code. Albumentations is a fast, GPU-friendly augmentation library purpose-built for deep learning training pipelines, offering a far broader and more composable set of augmentation operations — geometric, photometric, and more exotic augmentations such as Cutout and grid distortion — than either OpenCV or torchvision's transform module natively provides.

Library	Primary Use Case	Best Fit
OpenCV	Classical CV ops, video I/O, real-time capture	Feature detection (Day 28), live video pipelines
PIL / Pillow	Basic image manipulation	Simple load/resize/crop/convert tasks
torchvision / timm	Pretrained deep learning models & transforms	Transfer learning, benchmarking backbones (Day 30)
Albumentations	Fast, composable training-time augmentation	Deep learning training pipelines (Day 27)

💡 Use Case: Choosing a Library Under Time Pressure

A hackathon team needs to quickly prototype a "count people entering a room" system. Reaching first for OpenCV's built-in HOG-based pedestrian detector (Section 5.3's HOG, packaged as a ready-to-use classical detector) gets a working demo running in under an hour, with no training required at all.

A production team building the same feature for permanent deployment would instead reach for a fine-tuned deep detector (Day 31) via torchvision — slower to get running initially, but far more accurate and robust once trained — illustrating that "best library" is always a question of stage (prototype vs. production) as much as raw capability.

It is worth flagging explicitly that this section only introduces these tools at the level of "what each one is for" — Day 27 picks up exactly this same set of libraries and covers, in full operational detail, the specific preprocessing and augmentation operations each is actually used to perform, so that the tooling introduced here does not feel disconnected from its concrete, hands-on use in the very next day's material.

8 Evaluation Metrics in Computer Vision

Because the task taxonomy in Section 4 spans several structurally different kinds of output, no single evaluation metric applies across all of computer vision — a metric appropriate for classification is close to meaningless for segmentation, and vice versa. This section introduces each task family's standard metric only at the level needed to recognise it and understand roughly what it measures; the deep, worked-example treatment of each is deliberately deferred to the day where that task is covered in full.

8.1 Classification Metrics: Accuracy and Top-k Accuracy

For image classification, accuracy — the fraction of test images assigned the correct single label — is the natural starting metric, and top-k accuracy (counting a prediction correct if the true label appears anywhere among the model's k highest-confidence predictions, most commonly $k = 5$) is a standard relaxation used on datasets like ImageNet, where some categories are genuinely difficult to distinguish even for a careful human annotator — the top-5 error figures quoted for AlexNet in Section 6.1 use exactly this relaxed metric.

8.2 Detection Metrics: IoU and mAP

For object detection (Day 31), evaluation is considerably more involved, because a prediction must be assessed on both whether the predicted class is correct and how well the predicted bounding box overlaps the true one — a comparison quantified by Intersection over Union (IoU), which in turn feeds into precision and recall at a chosen IoU threshold, and ultimately into mean Average Precision (mAP), the field's standard single-number detection metric.

 Worked Example: Computing Intersection over Union (IoU)

Suppose a predicted bounding box and the ground-truth box overlap in a region of 40 square pixels (the intersection).

The predicted box covers 100 square pixels total, and the ground-truth box covers 60 square pixels total.

Union = predicted area + ground-truth area – intersection = 100 + 60 – 40 = 120 square pixels.

$\text{IoU} = \text{intersection} / \text{union} = 40 / 120 \approx 0.33$.

A common convention (used throughout Day 31) treats a detection as a "correct match" only if $\text{IoU} \geq 0.5$ against the ground truth — by that convention, this particular prediction would NOT count as a correct detection, despite the two boxes genuinely overlapping.

8.3 Segmentation Metrics: Pixel Accuracy, Dice, and IoU

For segmentation (Day 33), the same IoU concept is applied at the pixel level rather than the box level, alongside pixel accuracy and the Dice coefficient — a metric borrowed from and especially prominent in medical image segmentation, previewed here ahead of its central role in Day 37. Dice is closely related to pixel-level IoU ($\text{Dice} = 2 \times \text{IoU} / (1 + \text{IoU})$) but weights partial overlaps somewhat differently, and both metrics matter more than raw pixel accuracy in medical contexts specifically because pixel accuracy can be misleadingly high on a segmentation task where the target structure (a small tumour, for instance) occupies only a tiny fraction of the image — a model that predicts "background everywhere" can score a deceptively high pixel accuracy while being clinically useless.

Task Family	Primary Metric	Where It Is Covered in Full
Classification	Accuracy / Top-k Accuracy	This day, §8.1
Object Detection	$\text{IoU} \rightarrow \text{Precision/Recall} \rightarrow \text{mAP}$	Day 31
Segmentation	Pixel Accuracy, mIoU, Dice	Day 33 / Day 37
Generative Modelling	FID, Inception Score (not covered here)	Days 34–35

This summary table is worth returning to as a quick-reference throughout the rest of the module: whenever a new day introduces a metric, it is worth checking which task family it belongs to and confirming it matches the taxonomy row (Section 4) the day’s main technique is solving.

9 Real-World Applications & Ethical Considerations

9.1 Applications Across Industries

Computer vision’s practical footprint is genuinely broad, and it is useful to survey that breadth before this module narrows into individual technical topics. In healthcare, vision systems assist in diagnostic imaging, surgical navigation, and — as Day 37 covers at length — increasingly sophisticated medical image analysis. In security and public safety, vision underlies surveillance, access control, and screening systems. In retail,

it drives automated checkout, shelf-inventory monitoring, and customer-behaviour analytics. In autonomous vehicles, it forms one of the primary perception modalities alongside lidar and radar. In agriculture, it supports crop and weed identification, yield estimation, and livestock monitoring. This day's own practical project, PP12 (Face Mask Detection), sits squarely within the public-health-screening use case: an entry-point camera system that can flag whether an individual is wearing a mask correctly is a direct, deployable instance of the broader healthcare-and-security overlap this section describes.

Applications at Scale: Retail Shelf Monitoring

Large grocery chains deploy fixed or robot-mounted cameras that run object detection (Day 31) continuously across store shelves, automatically flagging out-of-stock items for restocking — a task previously done via slow, periodic manual walkthroughs.

This is a good illustration of Section 4's taxonomy in action: the business problem genuinely requires object detection's per-item localisation, not merely classification's "are shelves generally full" judgement — the same reasoning worked through explicitly in Section 4's worked example.

9.2 Ethical Considerations: Bias, Surveillance, and Consent

This breadth of deployment, especially anywhere a vision system makes or informs a decision about a person, carries a corresponding set of ethical obligations that this module treats as a running thread rather than a footnote — a thread that later days (Day 28 §9, Day 31 §9, Day 33 §9, and others) will each extend into the specific ethical considerations of their own topic, explicitly referencing back to this section as the thread's starting point. The most extensively documented concern is bias and accuracy disparity in facial analysis systems: multiple independent audits, most prominently the Gender Shades study (Buolamwini & Gebru, 2018), have found that commercial facial analysis systems achieve substantially lower accuracy on darker-skinned individuals and on women than on lighter-skinned men, a disparity traced in large part to training datasets that historically over-represented lighter-skinned faces.

Beyond accuracy disparities, widescale deployment of facial recognition and related tracking technologies raises direct surveillance concerns — the technical capability to identify and track individuals across cameras and locations at scale did not previously exist at this cost and speed, and its availability changes the practical calculus around consent, anonymity in public spaces, and the potential for misuse by both state and private actors. Consent and data provenance are a related concern specifically at the training-data stage: many large face datasets used to train and benchmark these systems were assembled by scraping publicly available photographs without the depicted individuals' knowledge or consent, a practice that has drawn increasing legal and regulatory scrutiny. Underlying all of these concerns is dataset bias more generally — a model can only be as representative as the data it was trained on, and unrepresentative training data produces unrepresentative, and often discriminatory, model behaviour no matter how sophisticated the underlying architecture.

⚠ Bias in Facial Analysis Systems

The Gender Shades study (Buolamwini & Gebru, 2018) audited three commercial gender-classification systems and found error rates as low as roughly 1% for lighter-skinned men, compared with error rates as high as roughly 35% for darker-skinned women — a disparity of over 30 percentage points within the same commercial products.

This kind of documented, systematic accuracy gap is why fairness auditing across demographic subgroups is treated in this module as a required part of evaluating any face-related system, not an optional add-on — a principle directly relevant to how PP12 should be evaluated once trained.

9.3 A Framework for Responsible Deployment

None of the concerns above are reasons to avoid deploying computer vision systems altogether — they are reasons to deploy them deliberately. A practically useful checklist, returned to throughout this module's later ethics sections, asks: was the training data collected with meaningful consent; has the model's error rate been measured separately across relevant demographic subgroups rather than only in aggregate; is there a clear, proportionate justification for the specific surveillance or tracking capability being deployed; and is there a human-in-the-loop review step for any high-stakes automated decision, rather than a fully automated action with no recourse. These four questions recur, in various forms, across nearly every ethics discussion in this module.

10 Research Frontiers**10.1 Foundation Models: CLIP, DINO, and SAM**

Looking beyond this module's own twelve-day arc, several research directions are actively reshaping what "computer vision" means as a field, and it is worth previewing them here even though their full technical treatment lies further ahead. Foundation models — very large models pretrained on broad, often internet-scale data and then adapted (or used directly, with no further training) across many downstream tasks — have become the dominant paradigm at the frontier of the field. CLIP (Contrastive Language-Image Pretraining) learns a joint embedding space for images and text simultaneously, enabling zero-shot classification into categories the model was never explicitly trained to recognise. DINO and its successors demonstrate that self-supervised pretraining — learning rich visual representations without any human-provided labels at all — can match or exceed the quality of representations learned via traditional supervised pretraining. SAM (Segment Anything Model), previewed already in Section 4, demonstrates the same foundation-model paradigm applied specifically to segmentation, accepting flexible prompts (points, boxes, or text) rather than being restricted to a fixed, pre-defined set of object categories.

10.2 Vision Transformers: A Structural Alternative to Convolution

Architecturally, vision transformers (ViT) represent a genuine alternative to the convolutional architectures that have dominated the field since AlexNet, replacing convolution's local receptive fields and weight

sharing (Section 6.2) with the self-attention mechanism originally developed for natural language processing, applied to an image divided into a grid of patches. This is a large enough shift that it earns its own dedicated treatment in Day 30, which this section deliberately does no more than set up here.

10.3 Multimodal Models and Self-Supervised Pretraining

Multimodal vision-language models extend the CLIP idea of a joint image-text embedding space into full generative and conversational capability — describing images in natural language, answering questions about their content, and following instructions that reference visual input — capabilities this module returns to at a foundational level once its computer-vision-specific content concludes. And self-supervised pretraining more broadly, of which DINO is one prominent example, continues to narrow the historical gap between supervised and unsupervised representation learning, with direct practical significance for any domain (medical imaging chief among them, per Day 37) where large labelled datasets are expensive or difficult to obtain but large unlabelled datasets are comparatively abundant.

Research Spotlight: Zero-Shot Classification with CLIP

CLIP is trained by contrasting 400 million (image, caption) pairs scraped from the internet, learning to pull matching image-text pairs together in a shared embedding space and push mismatched pairs apart — with no fixed, hand-labelled category list involved at any point in training.

The practical payoff: at inference time, CLIP can classify an image into an entirely new category simply by embedding a text description of that category (e.g. "a photo of a capybara") and finding the nearest image embedding — a genuinely different paradigm from Section 8.1's fixed-category accuracy metric, and a direct preview of the vision-language content this broader research programme returns to later.

Chapter Summary: Key Takeaways

Before moving into the assignment brief, it is worth consolidating the ground this day has covered, since each of these points becomes load-bearing for the material that follows in Days 27 through 37.

- **Vision is an ill-posed inverse problem** (§1.2): recovering 3D scene structure from a 2D image requires additional assumptions, whether hand-designed (classical era) or learned from data (deep learning era).
- **Images are $H \times W \times C$ tensors** (§2.1), and colour-space choice — RGB, HSV, YCbCr — is a genuine design decision with direct practical consequences, not just a storage convention.
- **CNNs take loose inspiration from biological vision** (§3) but differ fundamentally in their learning mechanism, and adversarial examples prove the two systems are not equivalent.
- **The task taxonomy** (§4) — classification, detection, segmentation, and beyond — is the map for the rest of this module's technical content (Days 29–35).

- **The classical pipeline** (§5) — detect, describe, match, classify — plateaued because hand-designed features cannot capture the full variability of real-world images, motivating the deep learning shift (§6).
- **Transfer learning** (§6.3) is the single most practically important consequence of the 2012 AlexNet result, and it is the technique this day's own assignment (PP12) puts directly into practice.
- **Ethical deployment** (§9.2–9.3) is a running thread through every remaining day of this module, not a one-time checkbox — fairness auditing, consent, and proportionality apply to nearly every technique covered from here forward.

Assignment / Practical Project Brief

PP12: Face Mask Detection

This project asks you to build a system that determines whether a person in an image is wearing a face mask, and to explicitly confront a real design choice in doing so: do you assume a cropped face is already given to you, or do you have to find the face yourself first? The two approaches below are not a "pick either one" menu — you are expected to understand, and ideally implement, both, and to reason explicitly about the trade-off between them, since that trade-off is the whole pedagogical point of this assignment.

A Approach A — Classification

This approach assumes a pre-cropped face image is already available as input, and reduces the problem to standard image classification: fine-tune a pretrained CNN backbone (MobileNetV2 and ResNet are both reasonable, Colab-T4-friendly choices, per Section 6.3's transfer-learning discussion) with a classification head over three classes — mask, no mask, and incorrectly worn mask. This is directly the transfer-learning workflow introduced in Section 6, and it is the simpler, faster path of the two: training is fast, the architecture is standard, and evaluation reduces to ordinary classification metrics (Section 8.1). Its limitation is equally direct — it says nothing about where a face is, so it requires an upstream face detector to be useful in any real, uncropped-image deployment.

```
import torchvision.models as models
import torch.nn as nn

# Approach A: transfer learning for 3-class mask classification
model = models.mobilenet_v2(weights="IMAGENET1K_V2")
model.classifier[1] = nn.Linear(model.last_channel, 3)
# classes: 0 = mask, 1 = no_mask, 2 = mask_incorrect
```

B Approach B — Detection

This approach solves the more realistic version of the problem: locate every face in an uncropped image and classify each one's mask status in the same pass. Two reasonable implementation routes exist — a lightweight SSD- or YOLO-style single detector trained directly on mask/no-mask/incorrect classes, or a two-stage pipeline that runs a dedicated face detector (Viola-Jones, per Section 5.5, or the more modern MTCNN) followed by the Approach-A classifier on each detected face crop. This is more realistic for an actual deployment scenario — an entry-screening camera feed, for instance — but is correspondingly more complex to train and to evaluate, since performance now depends on both localization accuracy and classification accuracy jointly (Section 8.2).

Dimension	Approach A (Classification)	Approach B (Detection)
Input assumption	Pre-cropped face given	Raw, uncropped image
Training speed	Fast	Slower (esp. end-to-end detector)

Dimension	Approach A (Classification)	Approach B (Detection)
Data / annotation needs	Cropped, labelled face images	Bounding boxes + labels
Deployment realism	Requires upstream face detector	Directly deployable on raw camera feed
Evaluation	Standard classification metrics (§8.1)	Detection metrics (mAP, §8.2, per Day 31)

C Datasets, Rubric, and Submission

Suggested datasets include the Face Mask Detection dataset on Kaggle (image classification format, well suited to Approach A) and the Face Mask Detection dataset with XML/YOLO-format bounding box annotations (well suited to Approach B) — both are small enough to train within a single Colab T4 session.

Rubric Component	Weight	What's Assessed
Approach A implementation	25%	Correct fine-tuning; sensible train/val split; reported metrics
Approach B implementation (or clear design)	30%	Correct detector training/pipeline; mAP or equivalent reported
Comparison table & analysis	20%	Accuracy, speed, data needs, deployment realism — mirroring §C's table above
Fairness check (§9.2)	10%	Error rate examined across visibly different demographic groups in the test set
Code quality & write-up	15%	Clean, documented code; clear written recommendation

Common pitfalls: forgetting the OpenCV BGR-vs-RGB channel order (Section 2.4) when mixing OpenCV-based face detection with a torchvision/PyTorch classifier; training Approach A on a dataset with a severe class imbalance between "mask," "no mask," and the comparatively rare "incorrect mask" category without any re-weighting or resampling; and, for Approach B, evaluating detection quality using classification-style accuracy alone rather than the box-aware metrics (IoU, mAP) that Section 8.2 introduces and Day 31 covers in full — a mismatch that will make your results difficult to interpret or compare against published baselines.

D Submission Checklist

- Approach A: fine-tuned classifier trained and evaluated with accuracy, precision, recall, F1
- Approach B: detector trained/pipeline built and evaluated with mAP (or a clearly justified proxy)
- Comparison table completed with your own measured numbers, not placeholders
- Short written recommendation: which approach fits which deployment scenario, and why
- Fairness check performed and documented, however small the available test set

- Code is documented, and a short README explains how to reproduce your results

End of Day 26 — ready for gap review and expansion into full compendium.